# Árvores

Universidade Estadual de Mato Grosso do Sul
Bacharelado em Ciência da Computação
Algoritmos e Estruturas de Dados II
Prof. Fabrício Sérgio de Paula

# Tópicos

- Introdução
- Definições e representações básicas
- Árvores binárias
- Árvores m-árias
- Percurso em árvores binárias
- Exercícios

# Introdução

- Estruturas sequenciais: não adequadas para todas as situações
  - Ex.: Banco de dados com inserções, remoções e buscas constantes
    - Lista com alocação sequencial:
      - Inserção: O(n)
      - Remoção: O(n)
      - Busca binária (lista ordenada): O(lg n)
    - Lista com alocação encadeada:
      - Inserção, remoção e busca: O(n)
    - Árvore binária balanceada:
      - Inserção, remoção e busca: O(lg n)

# Definições e representações básicas

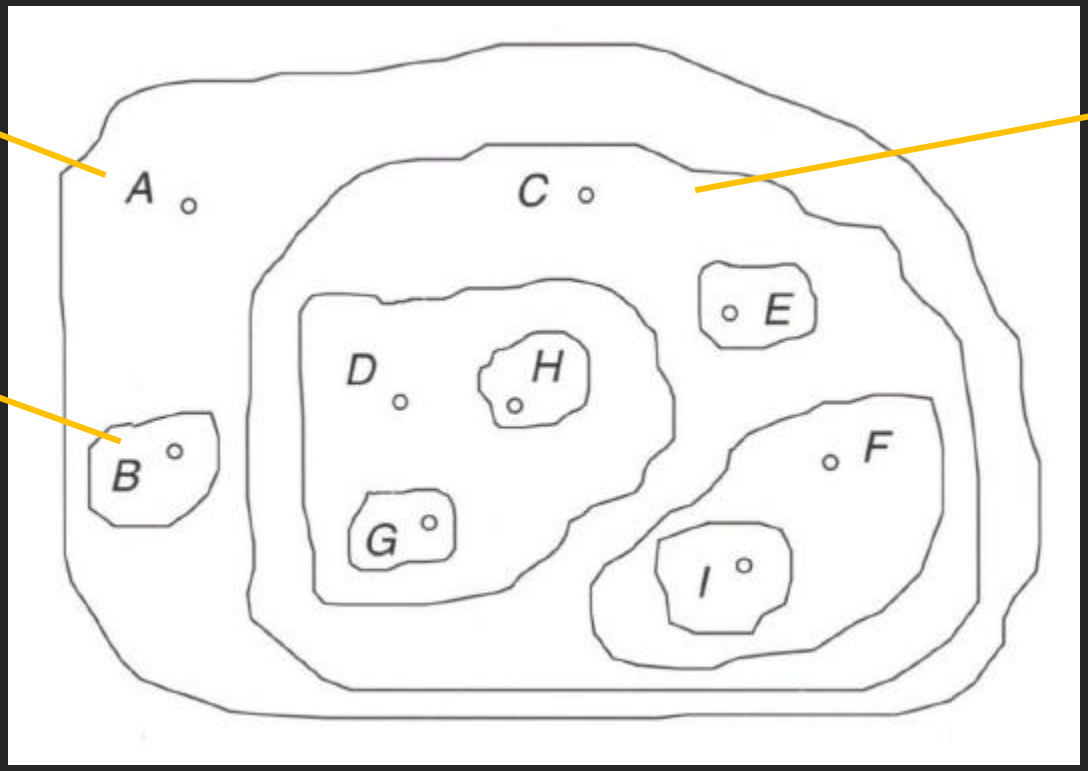
- Uma árvore T é um conjunto de nós (ou vértices) onde:
  - Se T = ∅, então a árvore é vazia
  - Senão:
    - Existe um nó especial r, que é a raiz de T
    - Os nós restantes formam um conjunto vazio ou são divididos em m ≥ 1 conjuntos disjuntos que são subárvores de r. Cada subárvore é também uma árvore

# Definições e representações básicas

- Representação de árvore pelo diagrama de inclusão

raiz

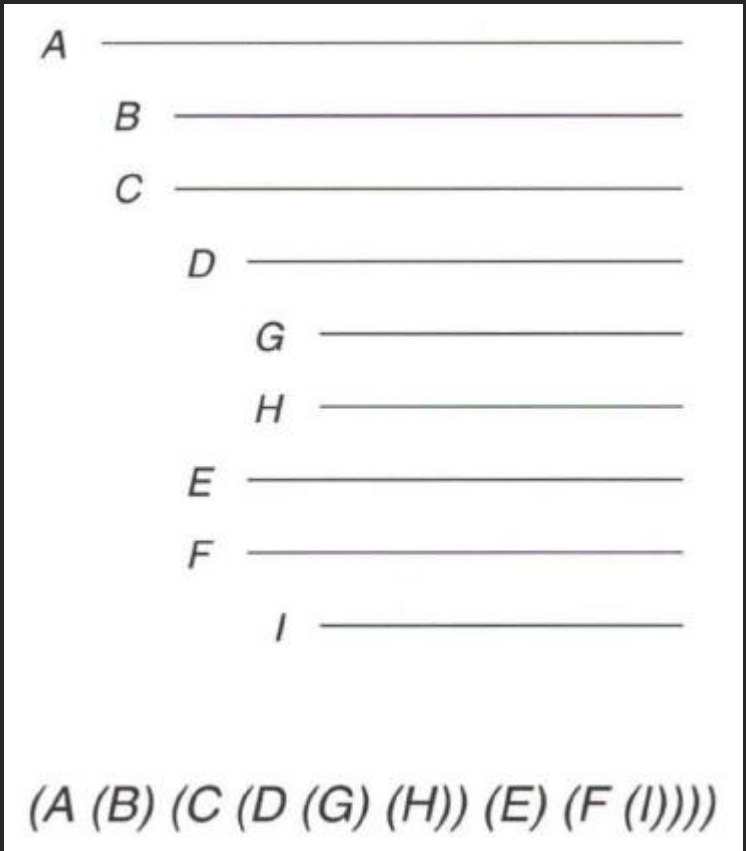
subárvore




subárvore

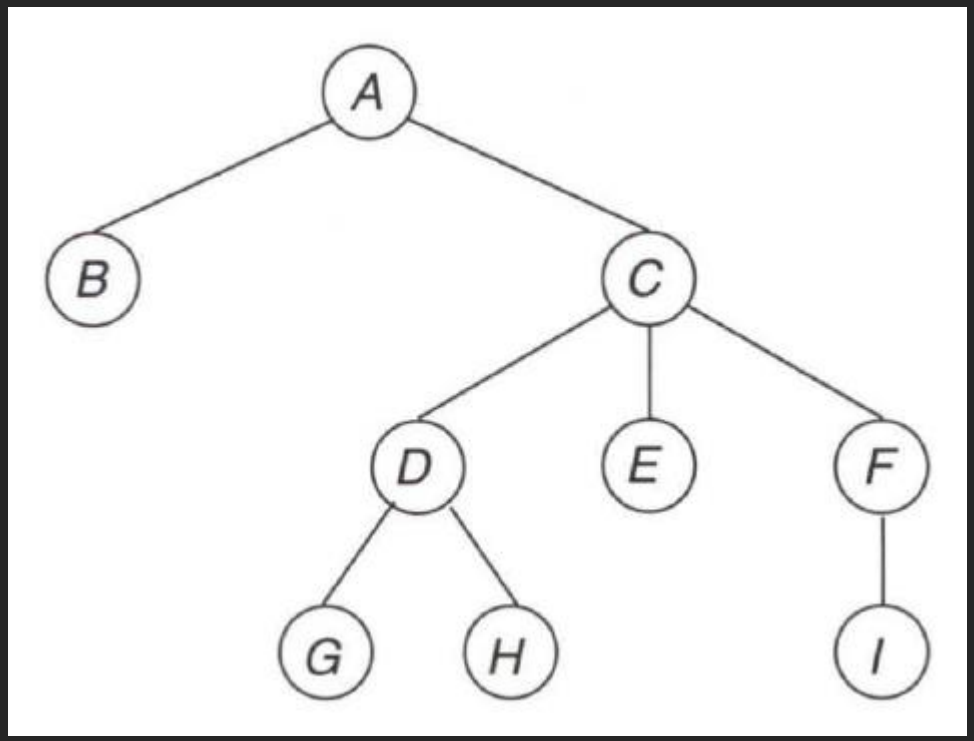
# Definições e representações básicas

- Representação por diagrama barras e parênteses aninhados



# Definições e representações básicas

- Representação hierárquica de uma árvore: mais utilizada



# Definições e representações básicas

- Seja v nó raiz da subárvore $T_v$ de T:
  - Filhos de v: nós raízes $w_1, w_2,\ldots,w_j$ das subárvores de $T_v$
  - v é pai de $w_1, w_2,\ldots,w_j$
  - $w_1, w_2,\ldots,w_j$ são irmãos
  - Se x pertence à subárvore $T_v$ então:
    - x é descendente de v
    - v é ancestral de x
    - Se $x \neq v$: x é descendente próprio de v e v é ancestral próprio de x
- Folha: nó sem descendentes próprios
- Nó interno (ou interior): não folha

# Definições e representações básicas

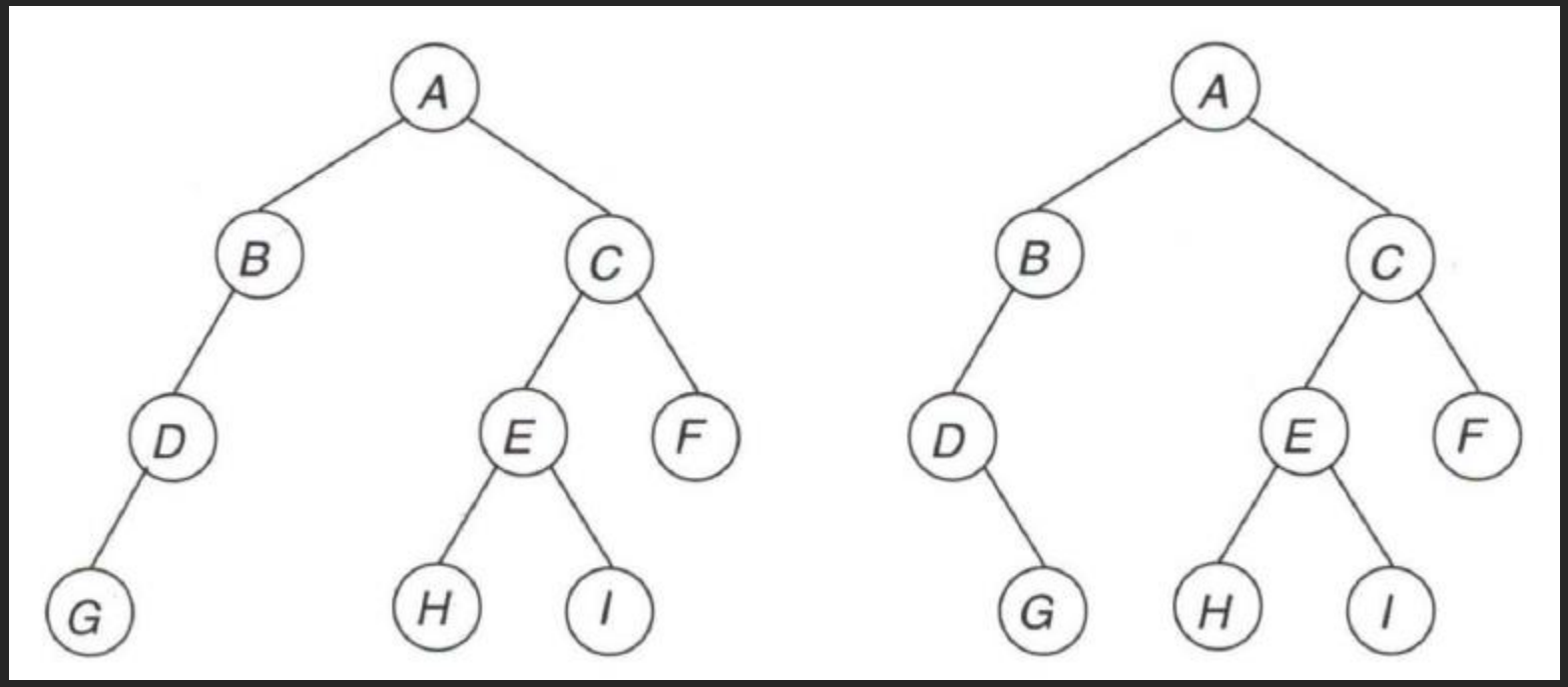
- Caminho na árvore: sequência de nós consecutivos $v_1, v_2, v_3, \ldots, v_k$ onde:
  - Para todo $1 \leq i < k$, $v_i$ é filho ou pai de $v_{i+1}$
  - k-1 é o comprimento desse caminho
- Nível de um nó v: número de nós do caminho da raiz até v
  - Nível da raiz: 1
- Altura de um nó v: número de nós do maior caminho de v até um de seus descendentes
  - Folhas têm altura 1
- Altura de uma árvore = Maior nível na árvore
- Árvore ordenada: filhos de cada nó estão ordenados (esq p/ dir)

# Árvores binárias

- Uma árvore binária T é um conjunto de nós onde:
  - Se T = $\emptyset$, então a árvore é vazia
  - Senão:
    - Existe um nó especial r, que é a raiz de T
    - Os nós restantes podem ser divididos em dois subconjuntos disjuntos $T_r^E$ (subárvore esquerda de r) e $T_r^D$ (subárvore direita de r), que também são árvores binárias
- Filho esquerdo (direito) de um nó v: raiz da subárvore esquerda (direita) de v

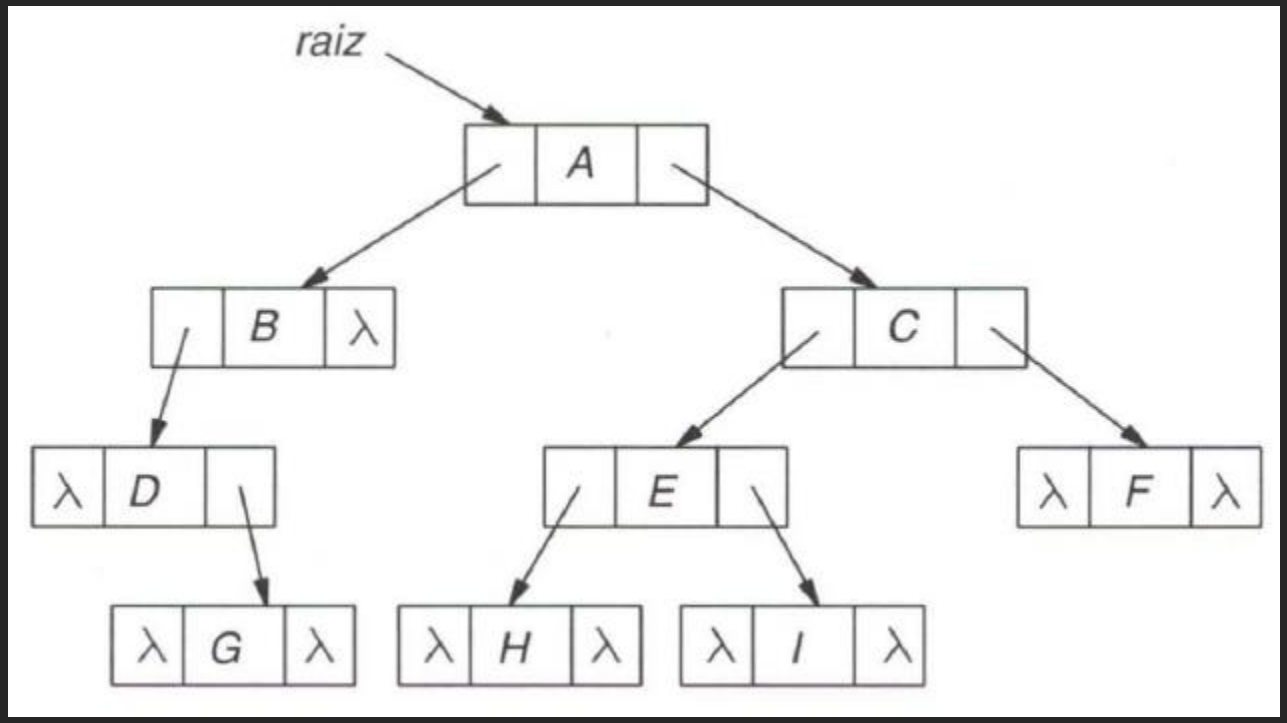
# Árvores binárias

- Exemplos de árvores binárias:
  - São iguais?



# Árvores binárias

- Ponteiros de uma árvore binária:

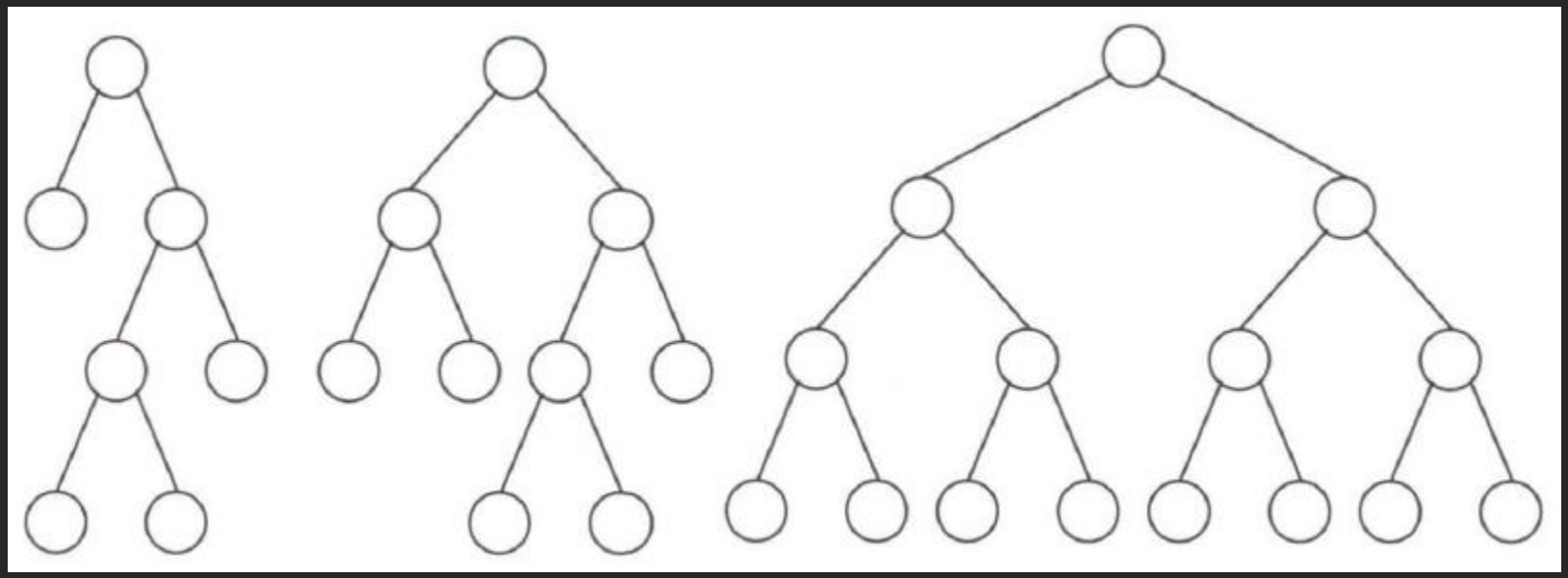


# Árvores binárias

- Tipos especiais de árvores binárias:
  - Árvore estritamente binária: cada nó possui 0 ou 2 filhos
  - Árvore binária completa: se nó v tem alguma subárvore vazia então v está no último ou penúltimo nível
    - Nós folhas estão no último ou penúltimo nível
  - Árvore binária cheia: se nó v tem alguma subárvore vazia então v está no último nível
    - Nós folhas estão no último nível
    - É completa e estritamente binária
  - Árvore ziguezague: nós não folha possuem apenas 1 filho

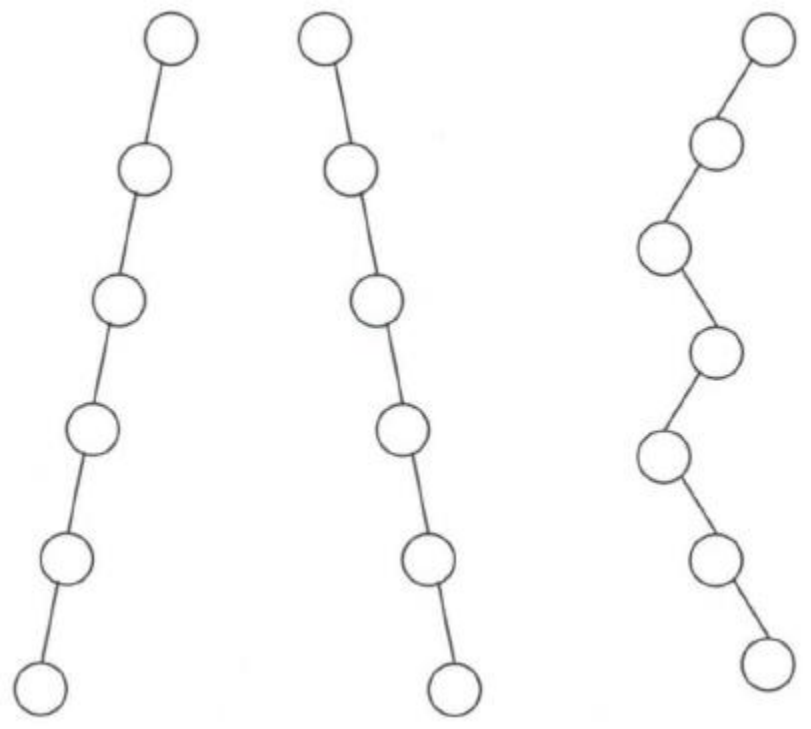
# Árvores binárias

- Exemplos de árvores: estritamente binária, completa e cheia



# Árvores binárias

- Árvores ziguezague:



# Árvores binárias

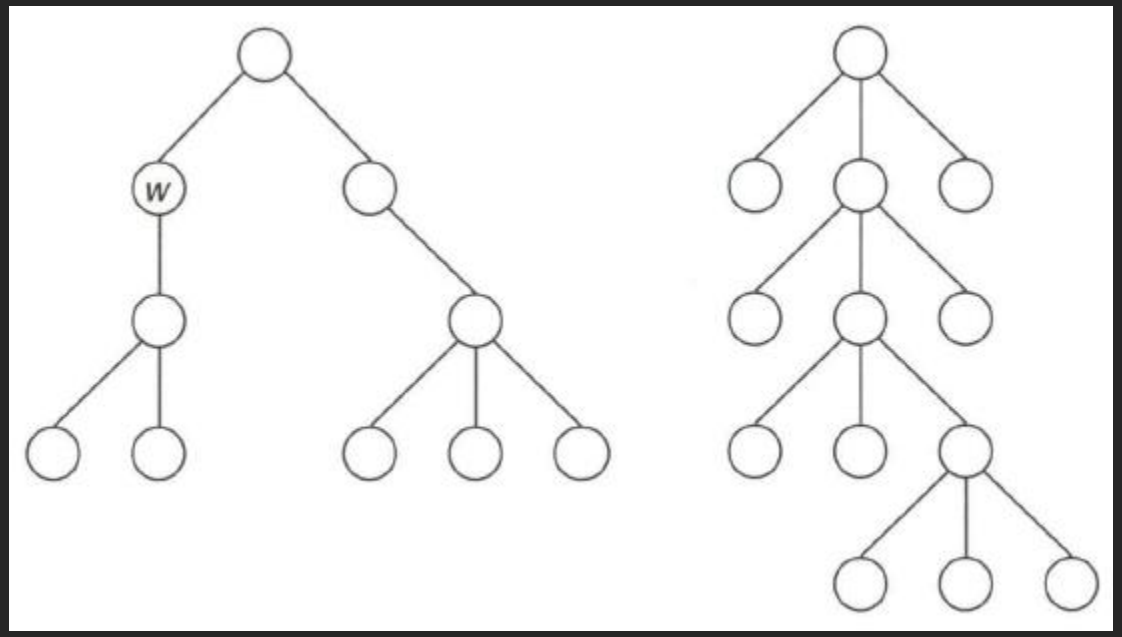
- Altura máxima e mínima de uma árvore binária com n nós?
  - Altura máxima: árvore ziguezague
    - Altura de árvore ziguezague com n nós: n
  - Altura mínima: árvore completa
    - Altura de uma árvore completa com n nós: $\lfloor \log_2 n \rfloor + 1$

# Árvores m-árias

- Uma árvore m-ária T, $m \geq 2$, é um conjunto de nós onde:
  - Se $T = \emptyset$, então a árvore é vazia
  - Senão:
    - Existe um nó especial r, que é a raiz de T
    - Os nós restantes podem ser divididos em m subconjuntos disjuntos, as i-ésimas subárvores de r, $1 \leq i \leq m$, as quais também são árvores m-árias

# Árvores m-árias

- Exemplos de árvores ternárias (3-árias):

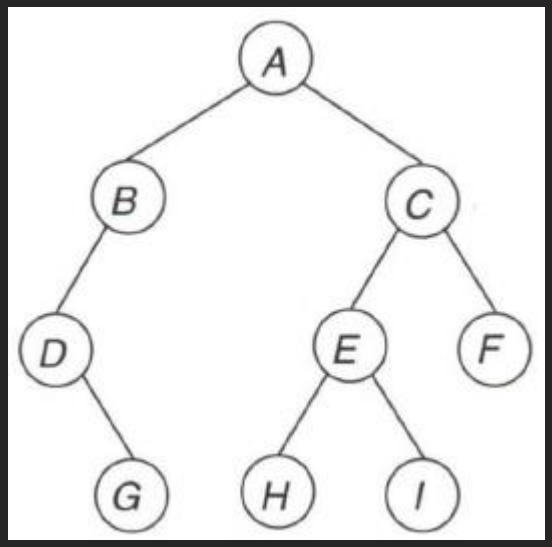


# Percurso em árvores binárias

- Percurso em uma árvore: visita sistemática a cada um dos nós da árvore
  - Cada nó é visitado exatamente uma vez
  - Exemplo de uso: impressão e atualização dos nós
  - Três principais percursos: em pré-ordem, em ordem simétrica, em pós ordem

# Percurso em árvores binárias

- Percurso em pré-ordem:
  1. Visitar a raiz;
  2. Percorrer subárvore esquerda em pré-ordem;
  3. Percorrer subárvore direita em pré-ordem.




**algoritmo 3.1:** Percurso em pré-ordem

**procedimento** $pre(pt)$

  $visita(pt)$

  **se** $pt \uparrow .esq \neq \lambda$ **então** $pre(pt \uparrow .esq)$

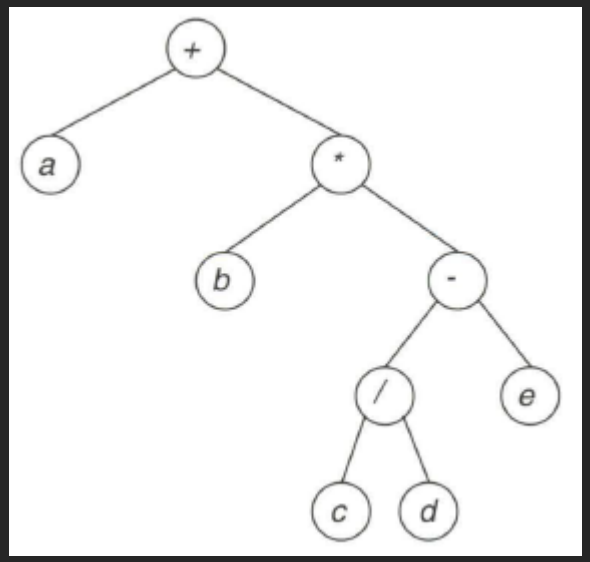
  **se** $pt \uparrow .dir \neq \lambda$ **então** $pre(pt \uparrow .dir)$

**se** $ptraiz \neq \lambda$ **então** $pre(ptraiz)$

Ordem de visita: A B D G C E H I F

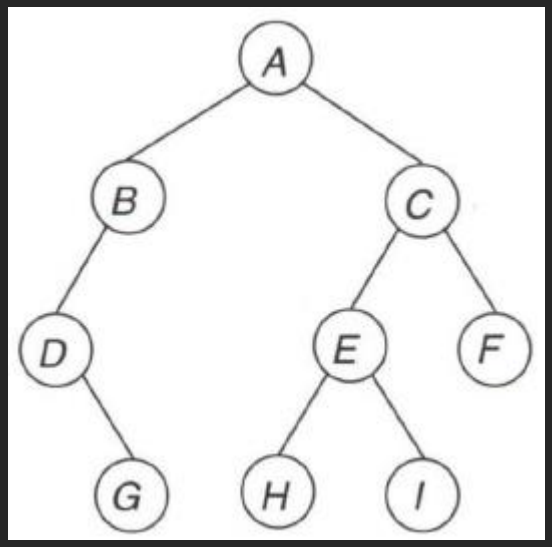
# Percurso em árvores binárias

- Aplicação do percurso em pré-ordem: notação polonesa
  - Expressão aritmética: a + b*(c/d – e)
    - Pode ser representada na forma de uma árvore binária
      - Percurso na árvore binária em pré-ordem: expressão na notação polonesa  + a * b - / c d e



# Percurso em árvores binárias

- Percurso em ordem simétrica:
  1. Percorrer subárvore esquerda em ordem simétrica;
  2. Visitar a raiz;
  3. Percorrer subárvore direita em ordem simétrica.




**algoritmo 3.2:** Percurso em ordem simétrica

**procedimento** $simet(pt)$

  **se** $pt \uparrow .esq \neq \lambda$ **então** $simet(pt \uparrow .esq)$

  $visita(pt)$

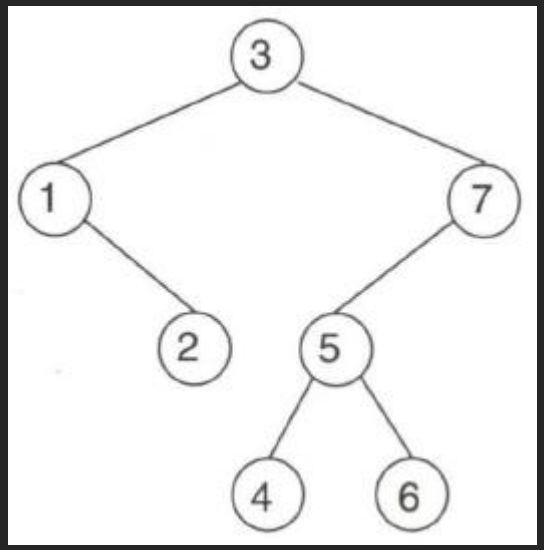
  **se** $pt \uparrow .dir \neq \lambda$ **então** $simet(pt \uparrow .dir)$

**se** $ptraiz \neq \lambda$ **então** $simet(ptraiz)$

Ordem de visita: D G B A H E I C F

# Percurso em árvores binárias

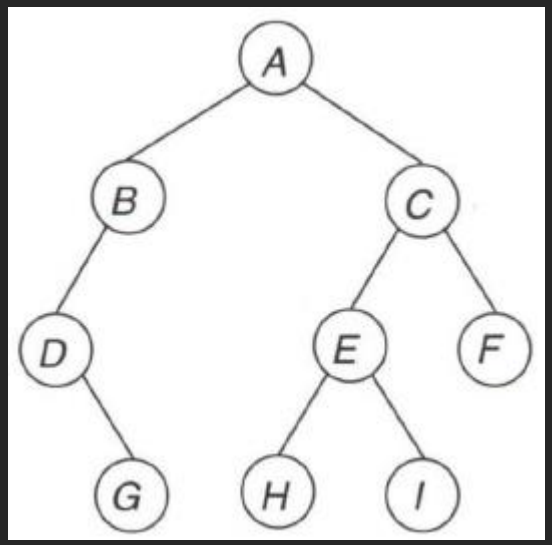
- Aplicação do percurso em ordem: apresentar os elementos de uma árvore binária de busca em ordem (ordenados)




- Ordem de visita: 1 2 3 4 5 6 7

# Percurso em árvores binárias

- Percurso em pós-ordem:
  1. Percorrer subárvore esquerda em pós-ordem;
  2. Percorrer subárvore direita em pós-ordem.
  3. Visitar a raiz;




**algoritmo 3.3:** Percurso em pós-ordem

**procedimento** $pos(pt)$

  **se** $pt \uparrow .esq \neq \lambda$ **então** $pos(pt \uparrow .esq)$

  **se** $pt \uparrow .dir \neq \lambda$ **então** $pos(pt \uparrow .dir)$

  $visita(pt)$

**se** $ptraiz \neq \lambda$ **então** $pos(ptraiz)$

Ordem de visita: G D B H I E F C A

# Percurso em árvores binárias

- Aplicação do percurso em pós-ordem: cálculo da altura de todos os nós
  - Procedimento visita() para cálculo da altura:

```
algoritmo 3.4: Cálculo da altura de um nó da árvore binária
procedimento visita(pt)
    se pt ↑ .esq ≠ λ então
        alt1 := (pt ↑ .esq) ↑ .altura
    senão alt1 := 0
    se pt ↑ .dir ≠ λ então
        alt2 := (pt ↑ .dir) ↑ .altura
    senão alt2 := 0
    se alt1 > alt2 então
        pt ↑ .altura := alt1 + 1
    senão pt ↑ .altura := alt2 + 1
```

# Percurso em árvores binárias

- Complexidade de cada um dos três percursos: O(n)
  - O procedimento é chamado n vezes
  - Cada vez tempo gasta tempo constante (considerando que visita é feita em tempo constante)

# Exercícios

- Livro Szwarcfiter:
  - 3.1, 3.2, 3.6, 3.7, 3.17, 3.18 a 3.20, 3.23, 3.25 a 3.27